



#SC-P-140725

— [Why we use MVVM?]

MVVM (Model View View Model)

--

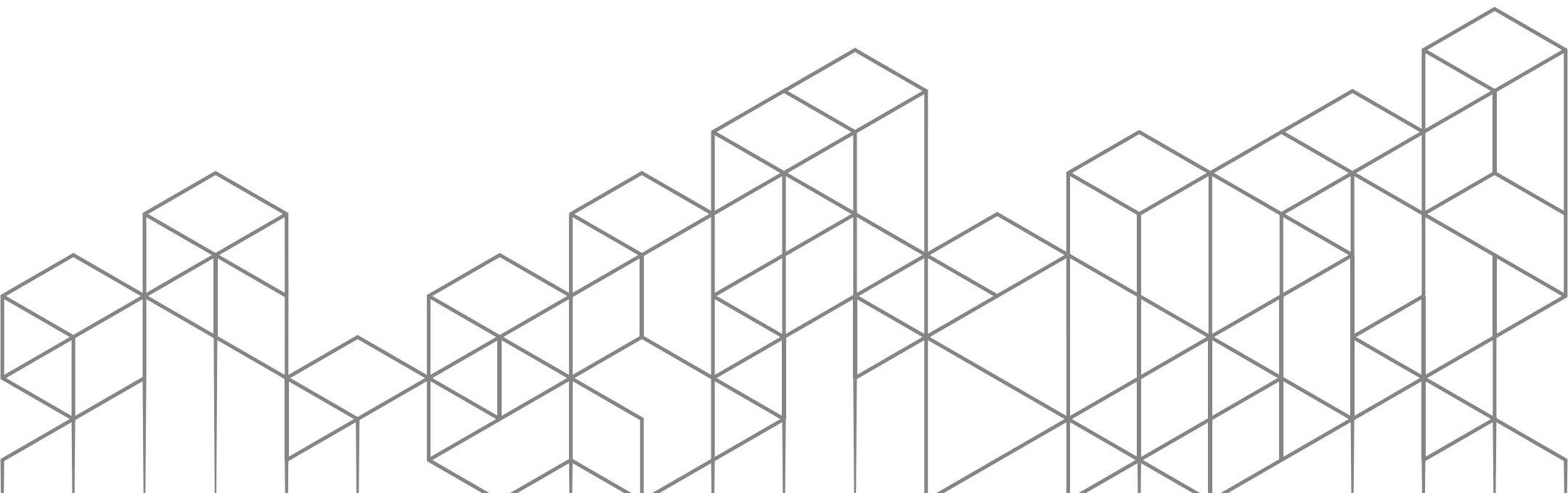
Designing Behavior, Not Just Views

MVVM is more than a UI pattern—it's a strategy for managing complexity. By separating behavior, state, and presentation, it allows teams to design how an application thinks, not just how it looks. The result is testable, scalable, and resilient code that mirrors real-world logic instead of UI quirks.

More than a pattern



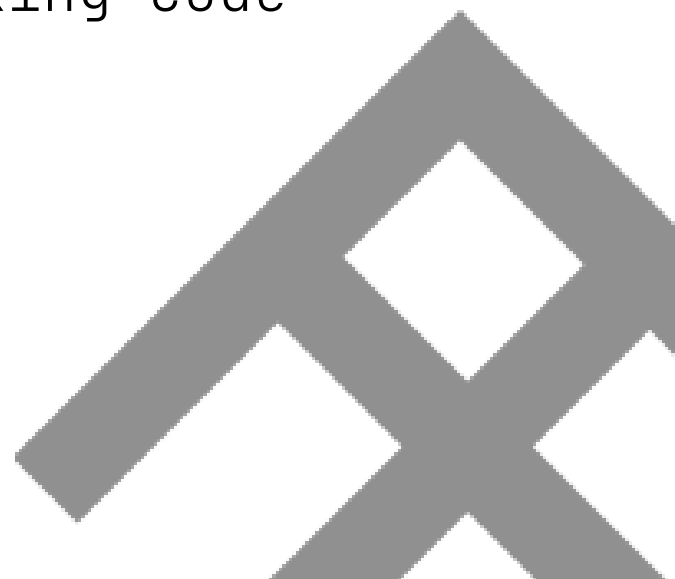
MVVM shapes how
software adapts and
survives.



Why Do We Use MVVM Architecture?



- **Because clarity scales better than complexity**
MVVM separates responsibilities, making large codebases easier to navigate, test, and maintain.
- **Because mixing layers breaks the architecture**
ViewModel keeps views clean and logic contained—protecting both user experience and code quality.
- **Because well-designed systems outlive features**
MVVM encourages sustainable thinking—code that adapts as products evolve.



Decoupled, Testable, Reliable – The Right ViewModel



■ Decoupled by design

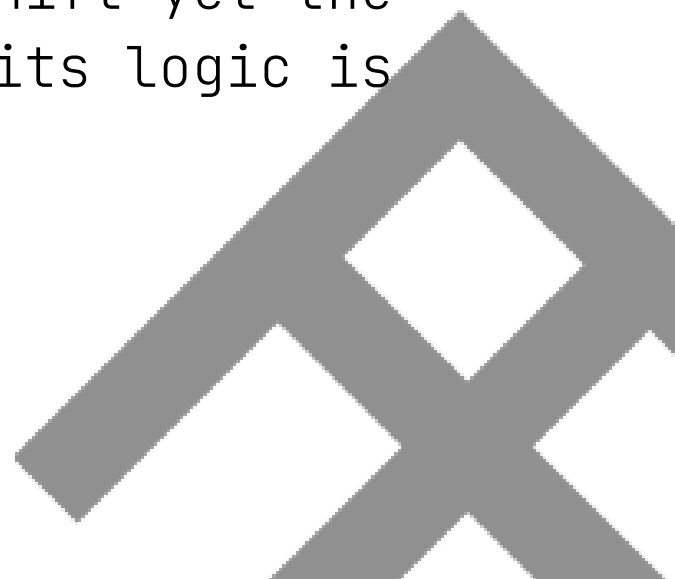
A proper ViewModel never touches UI code. It prepares data and exposes state—but stays unaware of how it's displayed.

■ Testable in isolation

Business logic, state transitions, and user interactions can be validated without rendering a single view.

■ Reliable across change

UI can be redesigned, themes can shift—yet the ViewModel remains stable, because its logic is cleanly separated.



The View Should Stay Dumb – On Purpose



■ Displays, but doesn't decide

The View shows what the ViewModel gives it. It doesn't calculate, fetch, or interpret logic.

■ Listens and delegates

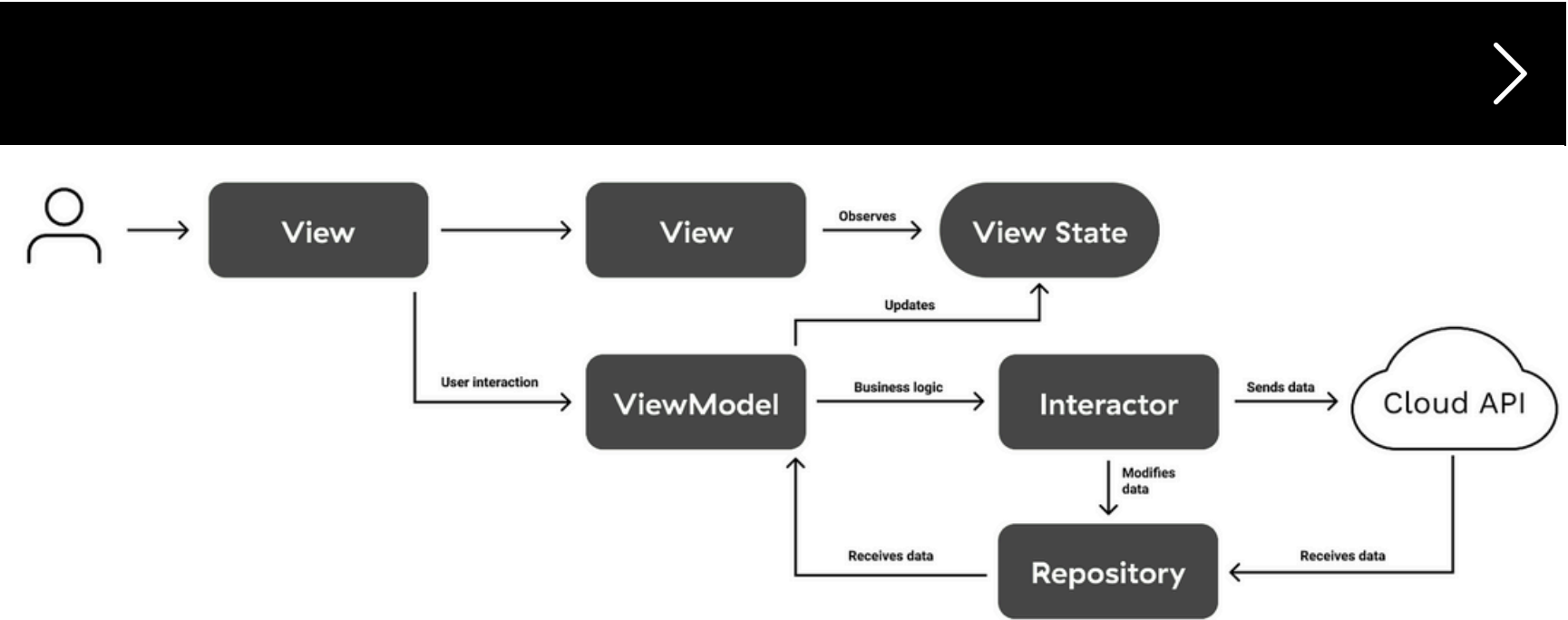
It observes changes and triggers actions—but always passes responsibility elsewhere.

■ Easy to redesign, easy to replace

When logic stays out, the View becomes flexible. You can rework the interface without rewriting behavior.



MVVM Extended: Purposeful Layers, Predictable Flow



We will demonstrate this with an example using the MVVM architecture in the Flutter framework. A more detailed implementation of this example can be found in our OSMEA project. Feel free to explore the repository and review our sample code.

OSMEA

MasterFabric represents the architecture of e-commerce mobile apps. Enterprise-level Flutter development.



From View to Logic – The MVVM Flow in Code



base_view_model.dart

```
abstract class BaseViewModelBloc<E, S> extends
Bloc<E, S> {
  BaseViewModelBloc(S initialState) :
super(initialState);

  void updateState(S newState) {
    emit(newState);
  }
}
```



From View to Logic – The MVVM Flow in Code



base_view.dart

```
class BaseView<V extends BaseViewModelBloc<E, S>, E, S>. extends StatefulWidget {  
  final Widget Function(V viewModel, BuildContext context, S state) builder;  
  final void Function(V viewModel, BuildContext context)? onViewModelReady;  
  
  const BaseView({  
    required this.builder,  
    this.onViewModelReady,  
    super.key,  
  });  
  
  @override  
  State<BaseView<V, E, S>> createState() => _BaseViewState<V, E, S>();  
}  
  
class _BaseViewState<V extends BaseViewModelBloc<E, S>, E, S>  
  extends State<BaseView<V, E, S>> {  
  late final V viewModel = GetIt.I<V>();  
  @override  
  void initState() {  
    super.initState();  
    widget.onViewModelReady?.call(viewModel, context);  
  }  
  @override  
  Widget build(BuildContext context) {  
    return BlocProvider.value(  
      value: viewModel,  
      child: BlocBuilder<V, S>(  
        builder: (context, state) {  
          return widget.builder(viewModel, context, state);  
        }  
      )  
    );  
  }  
}
```

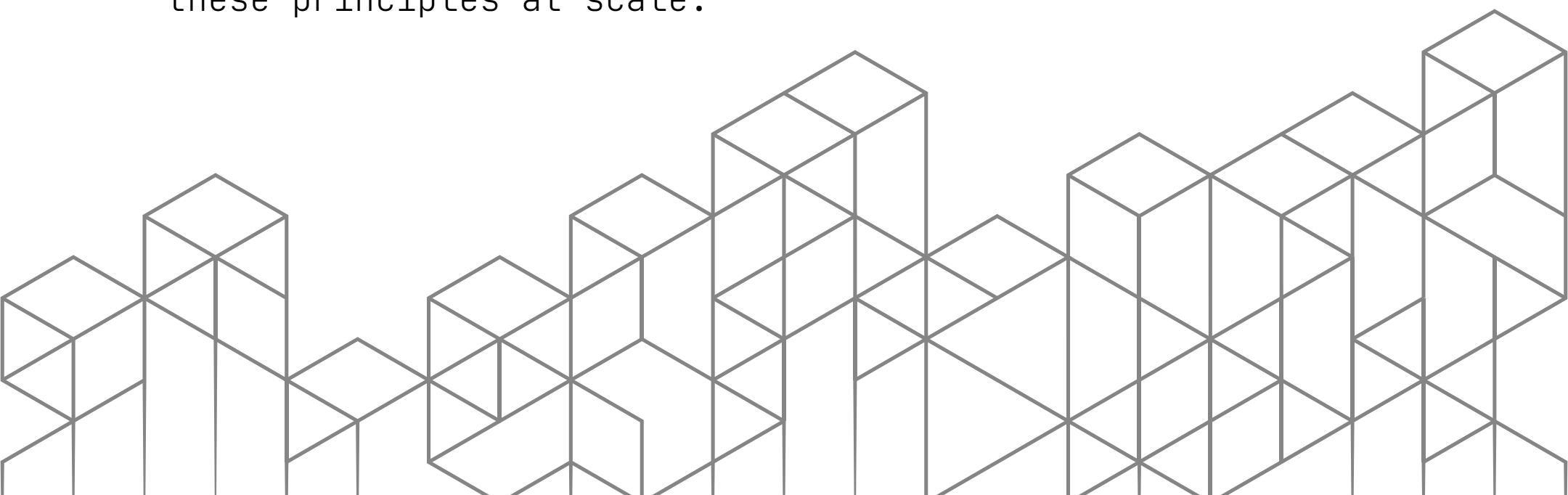


MVVM connects layers, logic, and longevity.

MVVM is more than an architectural pattern—it's a mindset for building resilient, future-proof systems. It brings clarity to complexity by isolating responsibilities across data, behavior, and presentation layers. When logic lives in the ViewModel, data in the Model, and display logic in the View, code becomes easier to test, evolve, and reason about. Architecture stops acting as overhead and starts enabling intentional design.

But the real power of MVVM lies in how it reshapes thinking. It enforces boundaries, encourages clean communication between layers, and builds systems that can survive redesigns, team changes, and scale. It's not just about how code is written—but how problems are framed.

We've adopted and extended this mindset in our OSMEA project, where MVVM meets real-world complexity with structure and discipline. Dive deeper into how we apply these principles at scale.





MasterFabric

Platform-Based Application
Development Studio